



UTM
UNIVERSITI TEKNOLOGI MALAYSIA

FACULTY OF COMPUTING

SEMESTER 2 2025/2026

SECP3133 HIGH PERFORMANCE DATA PROCESSING

SECTION 01

**PROJECT 1: OPTIMISING HIGH PERFORMANCE DATA PROCESSING FOR LARGE
SCALE WEB CRAWLERS**

LECTURER: PROF. MADYA. TS. DR. MOHD SHAHIZAN BIN OTHMAN

Group Triple A

Student Name	Matric No.
LAM YOKE YU	A23CS0233
NABIL AFLAH BOO BINTI MOHD YOSUF BOO YONG CHONG	A23CS0252
ANIS SAFIYYA BINTI JANAI	A23CS0049

Submission Date: 25 May 2026

Table of Contents

1. Introduction.....	4
1.1 Background of the Project.....	4
1.2 Objectives.....	5
1.3 Target Website and Data to be Extracted.....	5
2. System Design & Architecture.....	7
2.1 System Architecture.....	7
2.2 Tools and frameworks.....	8
2.3 Roles of group members.....	9
3. Data Collection.....	9
3.1 Crawling method.....	9
3.2 Records collected.....	12
3.3 Ethical considerations.....	13
4. Data Processing.....	14
4.1 Cleaning Methods.....	14
4.2 Data Structure.....	14
4.3 Transformation and Formatting.....	15
5. Optimisation Techniques.....	16
5.1 Method Used.....	16
5.2 Code Overview or Pseudocode of Techniques Applied.....	16
6. Performance Evaluation.....	19
6.1 Before vs after optimization.....	19
6.2 Time, memory, CPU usage, throughput.....	21
6.3 Charts and graphs.....	24
7. Challenges and Limitations.....	25
7.1 Challenges and solutions.....	25
7.2 Limitations of the solution.....	27
8. Conclusion and Future Work.....	27
9. References.....	28
10. Appendices.....	28

1. Introduction

This section provides an overview of the projects, including the project's background, objectives, and target website to be extracted. It becomes the foundational context necessary for understanding the scope and significance of the project.

1.1 Background of the Project

With the rapid development of online recruitment websites, a large amount of publicly available recruitment data continues to accumulate. With the evolution of the digital economy, platforms like Maukerja have become the main source of employment-related information with their structured and unstructured data. However, to successfully use such a large-scale data set, a reliable and efficient data processing process must be applied to cope with the core complexity and massive data of web crawler information.

There are significant performance bottlenecks found in the application of traditional data processing technology to large-scale web crawler data sets. These bottlenecks are particularly visible in terms of execution time, memory usage, and overall scalability, thus limiting the analytical efficiency of traditional methods. Therefore, it is crucial to evaluate and implement an alternative data processing framework that can provide better performance to manage data sets containing hundreds of thousands of records more effectively. This project effectively solves these problems by building a high-performance data processing process specifically for massive network crawler data sets. This project uses a variety of data processing frameworks, including Pandas, Polars and PySpark, in the implementation of the data processing process. Its main goal is to find the most efficient solution for large-scale analysis tasks in a stand-alone environment. The scope of the project covers the entire data processing life cycle, including automated web page crawling, data cleaning, preprocessing, performance benchmarking, and result visualization.

Automated web-crawling methods and Python-based frameworks are used to systematically collect recruitment information from the Maukerja website. A large data set containing more than 100,000 different recruitment records is used to evaluate the performance of each framework. Then, several key performance indicators are compared, including execution time,

CPU utilization, memory consumption, and data throughput. In addition, this project also explores how to strategically apply different optimization strategies to optimize the overall throughput, reduce resource consumption, and improve the processing efficiency of large-scale data analysis tasks.

1.2 Objectives

The objectives of this project are as follows:

1. Establish an automated web crawler system to systematically collect comprehensive job posting data obtained from the online recruitment website Maukerja.
2. The data sets collected through preprocessing and cleaning ensure the quality, consistency and relevance of the data for analysis and processing.
3. Build and evaluate multiple data processing frameworks in a single benchmarking environment, especially Pandas, Polars and PySpark.
4. Use key indicators such as throughput, memory usage, CPU utilization and execution time to evaluate and analyse the performance of each framework.
5. Determine the best data processing framework for managing massive network crawler data sets in a stand-alone computing environment.
6. Explain the effectiveness of optimization strategies in improving the performance of high-throughput data processing processes.

1.3 Target Website and Data to be Extracted

The target website of this project is Maukerja, a well-known online recruitment platform that mainly publishes job information from Malaysia and Singapore. The reason for choosing Maukerja as the main data source is its extensive coverage of the regional labour market, the variety of job categories and employment types, and the accessibility of its open job list page. We use the Python libraries BeautifulSoup and Requests for web crawling, which can automatically traverse and extract data from multiple pages and support the combination of various filter criteria.

The data acquisition process is divided into two independent crawling stages. The crawler first collects job list data from the paging search results and identifies the key features of each job

advertisement. For each unique record, in the second stage, more location information and detailed job descriptions are acquired. This approach is applied in order to reduce unnecessary queries and maximize the performance of the whole crawling process. The main data fields extracted from the Maukerja website include job ID, job title, company name, place of work with state and country where you work, salary value, employment status, job posting date, detailed job description and the corresponding job website. After the data collection was completed, a total of 106,081 unique job records were successfully collected, and they were preprocessed and benchmarked.

2. System Design & Architecture

The system is a data processing pipeline consisting of four main stages: web scraping, data preprocessing and cleaning, data processing using multiple libraries and frameworks, and performance visualisation.

The objective of the system is to collect job postings from the Maukerja website and to evaluate and compare the performance of different data processing libraries, namely Pandas, Polars, and PySpark.

2.1 System Architecture

The system consists of four sequential components. Figure 1 illustrates the overall system architecture.

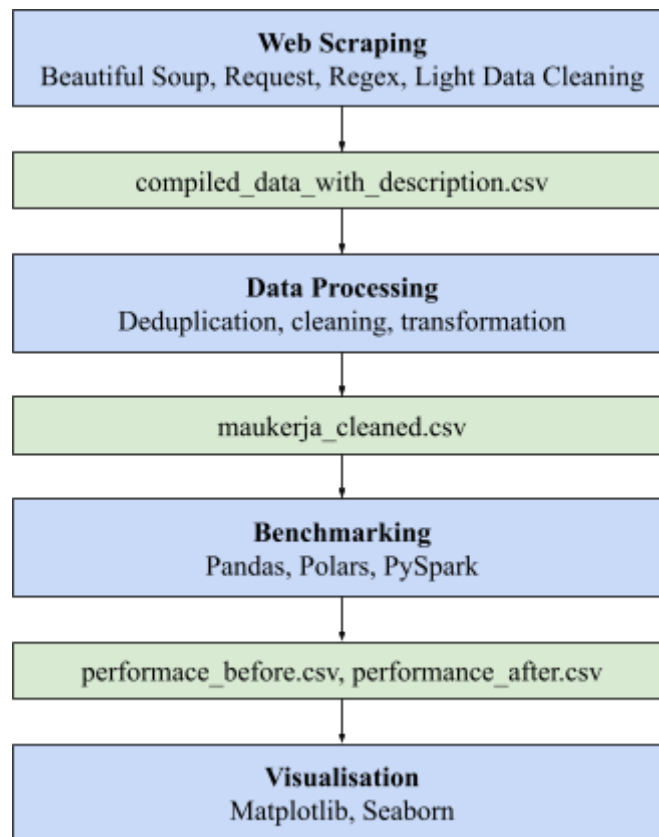


Figure 1: System architecture

Web Scraping

In this layer, data were collected from the Maukerja website. The HTML content of each webpage was parsed using BeautifulSoup, and relevant job-related fields were extracted. In addition, job identifiers were extracted using regular expressions.

The scraped data were subsequently merged, processed, and deduplicated into a unified dataset. Following this, each job listing was revisited to extract additional information, including job descriptions and detailed location data.

Data Processing

At this stage, the dataset was cleaned and standardised. The preprocessing operations included:

- Normalisation of salary values
- Standardisation of regional/state information
- Column renaming
- Removal of irrelevant or duplicate records

This stage ensured that the dataset was structured and suitable for performance benchmarking across different data processing libraries.

Benchmarking

In this phase, three different data processing frameworks were used, namely Pandas, Polars and PySpark. Each framework was used to perform identical computational operations in order to ensure a fair comparison.

The benchmarking operations included:

- Filtering job postings that contain salary information
- Counting job postings grouped by state
- Identifying top companies based on job posting count
- Classifying salaries into predefined salary bands

Performance was evaluated using four metrics:

- Execution time
- Memory usage
- CPU utilisation
- Throughput

Each experiment was executed three times and the results were averaged.

Visualisation

The benchmarking results were visualised using appropriate plots to compare the performance of each library across the defined metrics.

2.2 Tools and frameworks

Several tools and frameworks were used.

- Core programming language: Python
- Data storage: CSV, GZIP
- Web scraping libraries: BeautifulSoup, Requests
- Data manipulation libraries: Pandas, Polars
- Distributed processing framework: PySpark
- Data visualisation libraries: Matplotlib, Seaborn
- Version control and coordination: GitHub

2.3 Roles of group members

The following table shows the roles and primary responsibilities of each member.

Roles	Primary Responsibilities	Member
Group Leader and Data Engineer	• Coordinates the team • Oversees scraping • Manages data storage • Ensures progress	Lam Yoke Yu
HPC and Performance Analyst	• Designs and implements optimisation techniques • Runs benchmarks • Produces performance comparison chart	Nabil Aflah Boo
Analyst and Documentation Lead	• Performs data cleaning and analysis • Prepares the final report • Develops the presentation slides	Anis Safiyya

Table 1: Roles and responsibilities of group members

3. Data Collection

BeautifulSoup and request libraries were used in this process. Request was used to send HTTP requests to the target webpage and fetch its raw HTML content and BeautifulSoup was used to extract specific parts of the content using tags and classes.

3.1 Crawling method

The entire scraping process was iterative, consisting of multiple stages and feedback loops. The scraping pipeline first scrapes job listing pages and subsequently retrieves detailed job descriptions after deduplication of job listings. Figure 2 illustrates the overall workflow of the scraping pipeline.

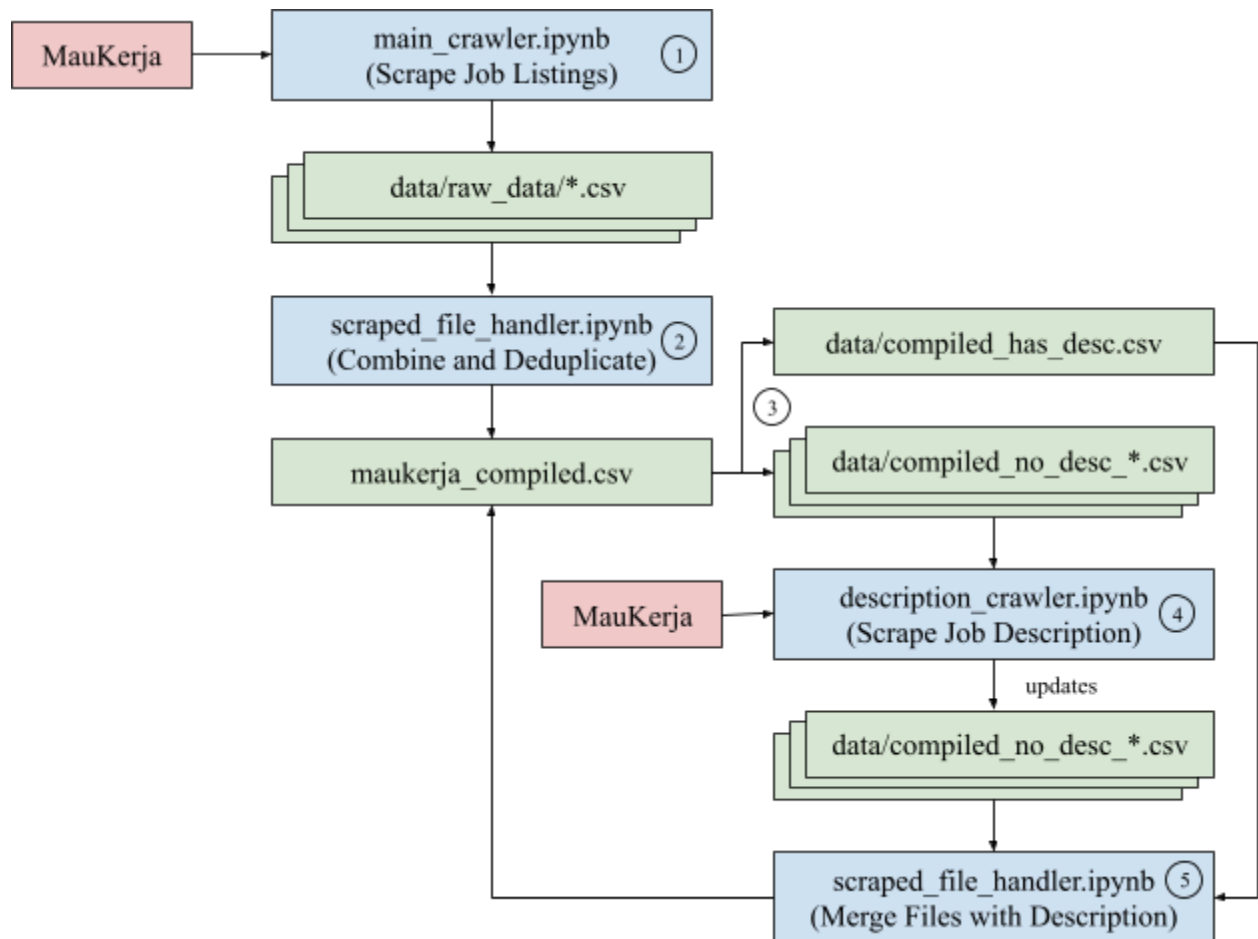


Figure 2: Overall workflow of the scraping pipeline

Step 1: Job Listing Page Crawling (`main_crawler.ipynb`)

This stage is responsible for scraping job listing pages from the Maukerja website. The website limits search results to a maximum of 167 pages per query, with each page containing up to 30 job listings, resulting in a maximum of 5,010 records per scraping session.

To maximize coverage, multiple filter combinations were applied, including job type, location, and posting date. The location filters covered all Malaysian states and districts, while the job type filters included full-time, part-time, internship, contract, volunteer, remote, and hybrid positions. In total, 138 locations and 7 job types generated 966 possible filter combinations.

However, it was observed that certain combinations contained very few or no records. Additionally, new job postings were frequently added to the platform. Therefore, additional scraping sessions targeting jobs posted within the previous 24 hours were conducted regularly.

Four categories of filter combinations were implemented:

1. Job type only

2. Location only
3. Combined job type and location
4. Jobs posted within the last 24 hours

Each filter combination generated a separate CSV file stored in the **data/raw_data** directory. For example, the “job type only” category produced one CSV file for each job type, while the “combined job type and location” category generated one CSV file for each filter combination.

The jobs were identified by selecting all elements enclosed within the **job-card-lite** class. The data within each job card was then parsed and stored.

The termination logic was revised several times, as it was observed that the website redirects to the first page when a requested page is unavailable, particularly in cases where the result set is very small. To mitigate duplicate data collection, the crawler was modified to terminate either when the maximum of 167 pages was reached or when the number of job listings returned on a page was less than 27, indicating that the last page of the filter had been reached. The threshold of 27 was chosen, instead of 30, to account for minor inconsistencies in page rendering and to avoid missing records due to partial page loads. This adjustment significantly reduced unnecessary page requests and improved overall scraping efficiency.

During the scraping process, job identifiers were extracted from the job URLs using regular expressions. However, some job URLs followed different structures and could not be processed using the same pattern. These cases were handled in the subsequent processing stages.

Step 2: Combine scraped files (scraped_file_handler.ipynb)

The raw data were compiled and deduplicated based on the job identifiers.

The CSV files stored in the **data/raw_data** directory were loaded into a DataFrame. For records without extracted job identifiers, the identifiers were determined using an alternative regular expression pattern different from the one used in Step 1.

If available, the **maukerja_compiled.csv** file was also loaded into a DataFrame. Both DataFrames were then combined and deduplicated based on the job identifiers. The resulting deduplicated DataFrame was subsequently saved as **maukerja_compiled.csv** in the **data** directory.

After processing, the CSV files were moved from **data/raw_data** to **data/raw_data_compiled** to prevent the reprocessing of previously scraped files, thereby reducing unnecessary processing time and memory usage.

Step 3: File separation (`scraped_file_handler.ipynb`)

The `maukerja_compiled.csv` dataset was separated into different CSV files containing:

- records with job descriptions (`compiled_has_desc.csv`)
- records without job descriptions

Records without descriptions were further divided into multiple chunked CSV files (`compiled_without_desc_*.csv`) to reduce processing load during the description scraping stage.

Step 4: Scrape Job Descriptions (`description_crawler.ipynb`)

The `compiled_without_desc_*.csv` files were loaded for detailed job description extraction. The description and detailed location of each job was scraped using the corresponding job URL. If a job description could not be found, the record was marked as “Job Closed”.

To reduce the risk of data loss during execution, the DataFrame was written back to the CSV file after every 50 processed records. This ensured that previously scraped data would still be preserved in the event of runtime errors or interruptions. Due to frequent file writing operations, input data were chunked in Step 3 to improve I/O efficiency and reduce processing overhead.

Step 5: Dataset Recombination (`scraped_file_handler.ipynb`)

All `compiled_*.csv` files were recombined and written back into `maukerja_compiled.csv`.

Step 6: Repeat Scraping Process

Step 1 was executed periodically to scrape newly posted job listings.

To prevent data corruption or data loss, Step 2 was executed only when Steps 3 to 5 were not running concurrently.

Step 7: Remove records without descriptions (`scraped_file_handler.ipynb`)

To prepare the data for data processing, the records without descriptions were removed.

3.2 Records collected

The job listing scraping process produced 1,106 CSV files containing approximately 1,349,403 records. The total runtime for job listing scraping was approximately 3,000 minutes (50 hours), while description scraping required approximately 7,500 minutes (125 hours), excluding interruptions caused by network failures and other operational issues.

A total of 106,081 unique records were collected. The data contained in each record are shown in the table below.

Attributes	Description
url	Job URL extracted from the href attribute or parent div
job_id	Job identifier extracted from the job URL using regular expression
title	Job title extracted from the job-card-lite container
company	Company name extracted from the companyName element
salary	Salary information extracted from the salary span element
job_type	Job type inside the jobtype-link anchor
location	Location inside joblocation-link anchors
experience_level	(Not consistently available in source data)
date_posted	Relative posting time extracted from the job listing page
skills	List of skills extracted from skill tag links
source_combo	Generated value of the scraping combination
scraped_location	Detailed job location extracted from job description page
scraped_description	Full job description extracted from job detail page

Table 2: Data description

3.3 Ethical considerations

Rate limiting: Randomised delays were introduced between requests during both Step 1 and Step 4 to reduce server load and minimise the risk of triggering anti-scraping mechanisms.

Public data usage: The dataset consisted only of publicly accessible job postings. No personally identifiable information (PII) of users or applicants was collected.

4. Data Processing

The data processing illustrates the cleaning methods, data structure, transforming and formatting. It describes the process of transforming the raw data collected through the web scraping pipeline.

4.1 Cleaning Methods

There are many data quality problems in the original data set obtained through web crawlers, so systematic data cleaning is required before analysis and processing. These problems include duplicate records caused by overlapping filter combinations during the crawling process, inconsistent formatting of classification fields and text fields, missing values of key attributes, and incomplete or missing job descriptions. This project implemented a structured data cleaning workflow to systematically solve all the above problems.

Firstly, based on unique job identifiers, identify and delete duplicate records to maintain data integrity and prevent the analysis results from being exaggerated. Before the benchmarking stage, we filtered out the records of missing valid job descriptions, because the lack of descriptive content will affect the representativeness and integrity of the data set. Salary fields are initially recorded in different formats in different job postings. We have cleaned and standardized them to make them conform to the unified numerical representation to facilitate salary-based analysis and classification tasks.

Regional and state name fields have also been standardized, and there are differences in naming rules for these fields, which is caused by inconsistent ways in which employers and recruiters enter location information on the platform. This standardized step is crucial to ensure the accuracy of location-based grouping and aggregation operations. Unrelated columns and records containing key missing information are deleted to reduce memory overhead and improve computing efficiency in the subsequent processing stage. For the absence of job identifiers in the main extraction field, an alternative extraction strategy using the regular expression mode is applied in the preprocessing process to recover the missing value. Finally, the column name is uniformly renamed to conform to a consistent and analytical descriptive format, thus improving the readability and usability of the data set.

4.2 Data Structure

This project adopts the CSV format as the main data storage structure in the entire data collection and processing process because the comma-separated value (CSV) format is easy to use, easy to port, and has extensive compatibility with the various data processing libraries used in this study. In the web crawling stage, each different combination of filters applied to the Maukerja platform will generate a separate CSV output file, which is stored in a specified raw data directory called `data/raw_data`. This modular storage method facilitates independent management and debugging of each crawl run.

After the crawling process is completed, each CSV file is merged into a unified data set called `maukerja_compiled.csv`, providing a single data source for downstream processing tasks. In order to adapt to the size of the data set and reduce the processing overhead of the detailed description capture stage, the merged data is further divided into multiple block CSV files, named `compiled_without_desc_*.csv` respectively. Stored files are compressed by GZIP to optimize disk space occupation and speed up data loading.

In the whole data processing process, the data set is represented by a tabular `DataFrame` structure, which is consistent with the paradigm supported by Pandas, Polars and PySpark. Given that the main focus of this study is high-performance memory data processing and comparative benchmarking of the framework for `DataFrame`, the use of relational database management system is deliberately excluded in the project design. This decision ensures that the performance measurement results obtained can reflect the inherent computing characteristics of each framework without interference from database input/output operations.

4.3 Transformation and Formatting

After the data cleaning stage, we have carried out a series of conversion and formatting operations on the data set to make it applicable to benchmarking and analytical queries. The salary value is decomposed and converted into two standardized numerical columns, named `salary_min` and `salary_max` respectively, for accurate salary-based screening, sorting and comparative analysis. These values are further grouped into predefined pay intervals to support the classification aggregation operation in the benchmark test experiment.

The state/provincial and location fields have been additionally reformatted to ensure that the naming rules of all records are consistent, thus improving the accuracy of geographical grouping and regional aggregation tasks. By directly converting string-based classification columns such as state/provinces and company names into classification data types in the Pandas framework, memory usage is significantly reduced. For Polars and PySpark frameworks, we apply pattern definition and explicit data type allocation when importing data to eliminate the overhead of runtime type inference and ensure deterministic data type processing.

In order to comply with the best practices of large-scale data processing, vectorized operations and column conversion strategies are always superior to progressive iterative processing methods. This method helps to improve the execution efficiency of all three frameworks. The final processed data set has a good structure and adopts a format-standard table data structure, which is suitable for performing various analytical operations, including aggregation, multi-condition filtering, connection operations and performance benchmarking, thus providing a reliable and representative basis for the comparative evaluation in this study.

5. Optimisation Techniques

The pipeline was benchmarked across 106,081 records of Malaysia job listing dataset from MauKerja website by utilizing various optimization methods. This was to measure their effectiveness on the cleaned dataset in executing five standard analytical queries which are jobs filtering by salary, counts of jobs posted by state, top companies exists, salary band classification, and throughput measurements. The data was read from a local CSV file and processed in memory for 3 selected libraries.

5.1 Method Used

The following data processing techniques are applied to evaluate their effect on performance:

1. **Pandas Baseline:** This acts as standard and does not apply any optimizations method on purpose as it reads all columns from the CSV without specifying the columns and performs analysis with default Pandas behaviour. It is used as the basis line for noting the improvements achieved from the other techniques.
2. **Pandas Optimized:** This approach implements specific optimizations at the code, directly in the Pandas framework such as loading targeted columns, specifying explicit data types, categorical encoding, and optimizing the same pipeline within the Pandas framework while keeping the libraries unchanged.
3. **Polars:** This approach uses Polars lazy evaluation Application Programming Key (API) by constructing and optimizing the entire query plan before starting to read the data from disk. Polars does the rest with a multithreaded, columnar, and Rust engine that automatically parallelises input and output, and computes across all the CPU cores.
4. **PySpark:** Apache Spark was used in local mode to explore a method that is designed for larger dataset and distributed environments. The dataset was then loaded via predefined schema into a Spark DataFrame, which then cached in memory, and each of the five analytical queries was reimplemented using Spark's DataFrame API and SQL Domain-Specific Language (DSL), allowing its Java virtual machine (JVM) based catalyst optimizer to handle the execution.

5.2 Code Overview or Pseudocode of Techniques Applied

The same approach across all methods was taken of loading the cleaned data from the local CSV file into memory at once and then each optimization technique was applied to the execution of the queries on the in-memory data.

a. **Pandas Baseline**

There are no restrictions or type specifications in Pandas DataFrame reading the entire CSV. This fully loaded DataFrame is then subjected to all five operations including salary filtering, grouping by state, company ranking, classifying salary ranges, and displaying the results.

Execution flow:

1. `pd.read_csv(CSV_FILE)` reads all the columns with runtime inference.
2. The `.notna()` data filters work on the `salary_min` and `salary_max` to do salary filtering.
3. `df.groupby('state')['job_id'].count()` is intended to count the numbers of job postings that are grouped by state.
4. `df.groupby('company')['job_id'].count()` provides a count of the jobs of the top companies.
5. The `salary_band` column is created by applying a custom Python function `classify_salary(val)` row by row using the `.apply()` method.
6. Each result from an operation is printed sequentially, and all performance metrics are captured by the benchmarking harness.

Data handling: All of the data in the entire DataFrame is in memory during execution except for the columns used in the pipeline. The `.apply()` call to classify the salary is done at the Python level and is executed row wise, which is not optimized by NumPy's vectorized engine.

b. **Pandas Optimized**

The same five queries are performed on Pandas with five targeted optimizations performed at the read stage and during computation to minimize I/O, memory usage, and CPU performance.

Execution flow:

1. `pd.read_csv(CSV_FILE, usecols=use_cols, dtype={ ... }, engine='c')` reads selected 7 columns that are defined ahead of time, without using any runtime type inference.
2. Low cardinality string columns (`state`, `company`, `country`) are changed to `pd.Categorical`, which uses compact integer codes instead of repeated string values.
3. On the float64 salary columns, `.notna()` is used to perform a vectorial operation of salary filtering.

4. `df.groupby('state', observed=True)` counts the number of postings per state, where `observed=True` means that it will not return an iteration of empty combinations of categories.
5. The row wise `.apply()` is replaced by a single pass with the vectorised binning to get salary_band from `pd.cut(df['salary_min'], bins=[ ... ], labels=[ ... ])`.
6. The same benchmarking metrics are used to measure the performances.

Data handling: Only the columns that are needed are kept in the memory. Categorical encoding optimizes how string columns are represented in memory and all transformations are done on NumPy arrays and not Python objects.

c. Polars

The pipeline is constructed using only Polars lazy API (`pl.scan_csv()`). The data is not obtained from the disk until the query is actually sent to disk through `.collect()` call, so that the query optimizer can modify and eliminate execution steps before performing any computation.

Execution flow:

1. No data is loaded yet as a lazy frame with pre-declared types is created with `pl.scan_csv(CSV_FILE, schema_overrides={ ... })`.
2. The operations of the filter, `lf.filter( ... )`, group_by and aggregation, `lf.group_by( ... ).agg( ... )`, and sorting, `lf.sort( ... )` are chained in the form of deferred expressions.
3. The `pl.when( ... ).then( ... ).otherwise( ... )` is a columnar expression tree compiled in Rust that will be evaluated across the whole column once.
4. For each `.collect()`, the optimizer pushes all predicate information down to the data, prunes the columns, and runs it across all CPU cores.
5. The performance metrics are gathered when all the `.collect()` have been completed.

Data handling: Polars skips the data on not used columns. Multithreaded I/O and computation are managed automatically for the user and operations remain cache efficient in columnar compatible memory format.

d. PySpark

A `SparkSession` is initialized and then configured to use local mode, with memory, shuffle partition and Arrow serialization settings set appropriately. The dataset is then loaded into a Spark `DataFrame` with a predefined `StructType`, and after that the data is

cached in memory for subsequent analyses, which are performed with Spark's DataFrame API and SQL DSL.

Execution flow:

1. The `SparkSession.builder.master('local[*]')` option uses all available cores on a machine, and `spark.sql.shuffle.partitions` is set to 4 to prevent the default 200 partitions overhead on a single machine.
2. `spark.read.schema(schema).csv(CSV_FILE).cache()` reads the data from a CSV file without the 2 pass inferSchema scan, and keeps the base DataFrame in memory.
3. `sdf.count()` is used to materialize the data in the cache to prevent the reparsing of the CSV when the subsequent operations are being run.
4. `sdf.groupBy('state').agg(F.count( ... ))`,
`sdf.groupBy('company').agg( ... )`, and
`sdf.withColumn('salary_band',`
`F.when( ... ).otherwise( ... ))` are Spark SQL queries and being compiled and optimized before it is executed.
5. With Apache Arrow serialization (`arrow.pyspark.enabled=true`), `.toPandas()` returns the results to Pandas for display with a minimal serialization cost.
6. Once all queries have been executed, `sdf.unpersist()` will remove the cached DataFrame from memory.

Data handling: Data is stored in the distributed memory structures of Spark, which are spread across the cores at hand. All transformations are lazy and only run when an action `.count()` and `.toPandas()` are triggered. All computation takes place in the JVM based execution engine and Python serves as the orchestration layer.

6. Performance Evaluation

This section compares the four processing methods quantitatively based on four performance metrics which are execution time, memory usage, CPU utilization and throughput. All methods were run three times on the same dataset and the average of the three runs is reported to improve accuracy in light of runtime variability.

6.1 Before vs after optimization

Three trials were performed for each method using the same record of dataset and the average were used for comparison between 3 libraries. The results are displayed in terms of four important metrics which are execution time, memory used, CPU utilization, and throughput.

Method	Run	Time (s)	CPU (%)	Memory (MB)	Throughput (rec/s)
Pandas baseline	1	7.5852	78.0	362.89	13,985
	2	5.8972	98.7	362.89	17,988
	3	5.4582	99.4	362.90	19,435
Average		6.3135	92.0	362.89	17,136
Pandas optimized	1	2.8967	100.01	32.82	36,621
	2	3.4061	98.5	32.82	31,144
	3	2.8973	99.6	32.82	36,614
Average		3.0667	99.4	32.82	34,793
Polars	1	1.9205	185.3	0.08	55,236
	2	2.3888	185.4	0.04	44,407
	3	2.2982	114.4	0.04	46,159
Average		2.2025	161.7	0.06	48,601
PySpark	1	5.5027	2.5	0.22	19,278
	2	6.3144	2.2	0.22	16,800
	3	5.4810	2.6	0.21	19,354
Average		5.7660	2.4	0.22	18,477

Table 3: Full Benchmark Results

In comparing the performance of Pandas baseline with the other three optimized methods, some consistent and measurable improvements can be observed in all the parameters. Overall, the code optimizations that are built into Pandas by conducting column pruning, explicit type annotation, categorical encoding, and vectorised binning brought the execution time down from 6.314s to 3.067s, and memory usage from 362.9 MB to 32.8 MB, resulting in a 2.06x speedup and over doubling the throughput of the code, which from 17,136 rec/s to 34,793 rec/s. Polars showed the highest overall result at 2.87x speedup with the fastest execution time of 2.203s, the least memory used of 0.1 MB, and it has the highest throughput of 48,601 rec/s. This is due to the fact that Polars lazy query planner defers computations and optimises them prior to running, its multithreaded columnar engine and its automatic parallelism across all available CPU cores, with the only way exceed saturation of 100% is to actually exceed the number of parallel cores on the

machine.

While the most architecturally advanced of the four methods, PySpark is not the fastest with the average execution time of 5.766s and a speedup of just 1.09x compared to the unoptimized Pandas baseline. This is due to well-known properties of distributed systems where the JVM initialization, Spark session start up, and the serialization of Python to JVM takes up a lot of resources at small and medium data size, and the benefits of PySpark parallelism are limited. It turns out that optimization methods can significantly boost performance, but it is able to do so as much as the workload and system configuration allows. These techniques are going to need larger data volumes or more computationally demanding operations to overcome the overhead of the technique.

6.2 Time, memory, CPU usage, throughput

To assess whether any of the execution methods actually affected performance, four metrics are commonly used to benchmark the performance. Three trials of each execution method were performed with each approach, strategy, and average values were obtained to ensure reliable and consistent execution of methods. This can reduce variability from fluctuations in the system and give a more realistic measurement of the performance of each method.

- **Execution Time (seconds)**

This is the part where the time required to process and return the result of a pipeline runs on a system. The shorter the execution time, the better the system will respond and the better the overall system performance. The table below shows the mean time taken for each method and averaged over three runs.

Run	Pandas baseline	Pandas optimized	Polars	PySpark
1	7.5852	2.8967	1.9205	5.5027
2	5.8972	3.4061	2.3888	6.3144
3	5.4582	2.8973	2.2982	5.4810
Average	6.314	3.067	2.203	5.766

Table 4: Execution Time (seconds)

Based on this data, it can be seen that Polars was always able to execute the fastest for all three runs. For instance, in Run 1, Polars was able to finish the pipeline in 1.9205 seconds while the Pandas baseline did in 7.5852 seconds. Polars on average is able to cut the execution time by around 65% from the unoptimized baseline. PySpark was the second

longest running framework but it was the most architected advanced framework, which is probably due to the fact that JVM initialization and serialization overhead dominates at this dataset size.

- **Memory Usage (MB)**

Memory is the amount of RAM used while the pipeline is running. The less memory required, the more resource efficient the method, and it is important especially for handling large datasets and many pipelines at the same time. The average memory usage of each of the methods is shown in the following table.

Run	Pandas baseline	Pandas optimized	Polars	PySpark
1	362.89	32.82	0.08	0.22
2	362.89	32.82	0.04	0.22
3	362.90	32.82	0.04	0.21
Average	362.9	32.8	0.1	0.2

Table 5: Memory Usage (MB)

From the table, Pandas baseline used by far the most memory at 362.9 MB on average which it imported the entire CSV file with all columns and stored the string fields as Python object arrays. With just some minor changes to the code, Pandas optimized cut this down to 32.8 MB using only column pruning and categorical encoding. Both libraries have reported near zero PySpark visible memory, namely 0.1 MB for Polars, and 0.2 MB for PySpark, as both libraries allocate their working memory outside of Python heap space and it is not exposed via the usual process level measurement.

- **CPU Utilization (%)**

This measures the ratio of CPU resources consumed during execution. If the value is greater than 100%, it means that the method is using more than one core simultaneously. The average CPU utilization recorded for every method is given in the following table.

Run	Pandas baseline	Pandas optimized	Polars	PySpark
1	78.0	100.1	185.3	2.5
2	98.7	98.5	185.4	2.2

3	99.4	99.6	114.4	2.6
Average	92.0	99.4	161.7	2.4

Table 6: CPU Utilization (%)

Based on the table, it's clear that Polars is the only approach to be able to make it past single-core saturation, as it averages 161.7% CPU utilization over the three executions, indicating real parallelism with multiple threads. They both used only one core of the processor, with approximately 92-99% of utilization. PySpark on the other hand, had the lowest Python visible CPU utilization at 2.4% since all the actual processing is done inside JVM executor threads, which are not visible at the Python level. The Python visible CPU utilization remains low as the Python driver process is mostly in idle mode.

- **Throughput (records/sec)**

Throughput shows the number of records per seconds that are being processed, and is a direct reflection of the parallelism and execution speed. The higher the throughput, the more efficient the pipeline. The average of the throughput for each method is shown in the table below.

Run	Pandas baseline	Pandas optimized	Polars	PySpark
1	13,985	36,621	55,236	19,278
2	17,988	31,144	44,407	16,800
3	19,435	36,614	46,159	19,354
Average	17,136	34,793	48,601	18,477

Table 7: Throughput (records/seconds)

From the table, it shows that Polars is the most efficient at 48,601 records per second on average, with records being processed almost three times as fast as the Pandas baseline. Pandas optimized also demonstrated a significant increase, over double the baseline throughput to 34,793 records per seconds, with only vectorized operations. PySpark, however, had only a marginal gain in performance of 18,477 records per seconds, which means that at this scale of data, the overhead of PySpark ate up a substantial amount of the total time budget of each run.

6.3 Charts and graphs

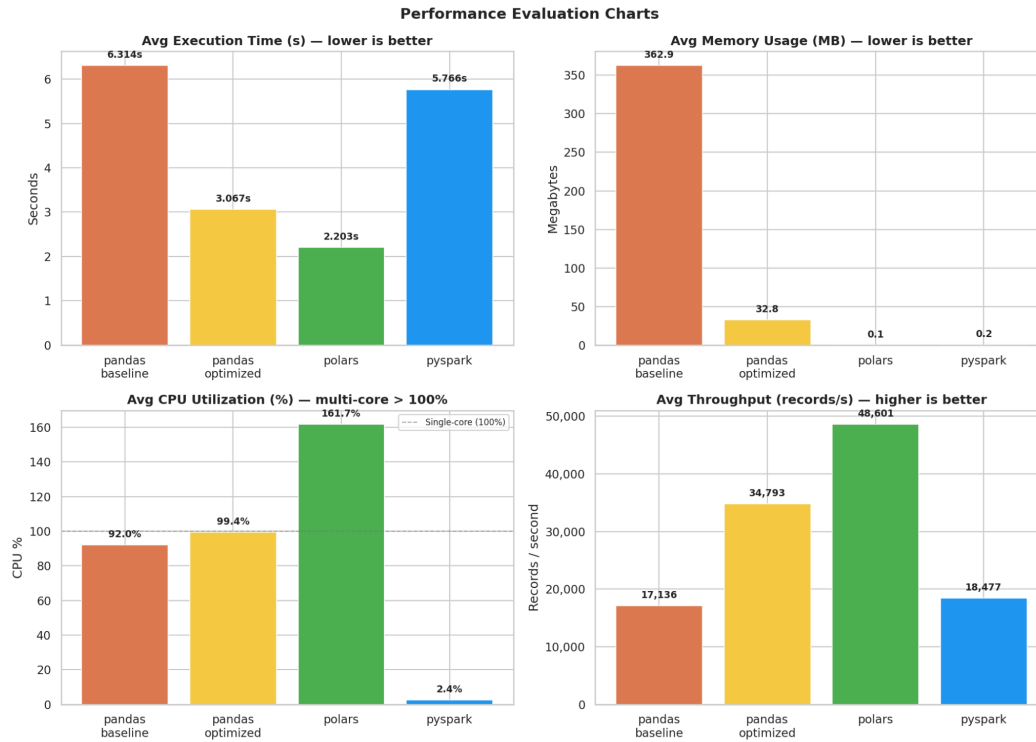


Figure 3: Performance Evaluation Charts

Figure 3 shows all the performance charts of each method in terms of the four dimensions of pipeline efficiency. The speed advantage of Polars is clearly highlighted in the execution time chart, where the reduction in processing times is significant compared to the baseline, and PySpark's bar is still much higher than expected despite the distributed architecture. It is evident from the memory usage chart that Pandas optimized drastically reduced memory usage, while Polars and PySpark almost used none at all compared to the clear outlier of the Pandas baseline memory usage at 362.9 MB. The CPU utilization chart is interesting as it shows only Polars to use up more than 100%, making it visually clear that it is parallel computing across multiple cores, whereas PySpark's bar is near to 0% as its computation is invisible to the Python level monitoring. The throughput chart further confirms the overall ranking, with Polars, Pandas optimized, PySpark, and the Pandas baseline, and shows that the ordering of throughput is stable and consistent for each of the three benchmark runs, suggesting reproducible results.

7. Challenges and Limitations

There were some challenges that were faced throughout the project.

7.1 Challenges and solutions

During data scraping, there were many problems that existed, causing revision of the scraping logic. The following table shows the problem faces, the solution implemented and the future improvements that can better mitigate the problem.

Problem	Solution	Future improvements
Regular Expression Issues Job IDs and URLs were not captured correctly. The initial code would set both <code>job_id</code> and <code>url</code> to <code>NA</code> if the URL did not match the regular expression. It was later observed that there were two different URL patterns in the job URLs. Initially, the scraping code was designed to remove duplicates during execution, and the failure to extract job IDs resulted in significant data loss.	The job URL is now captured and saved regardless of whether the job ID is successfully extracted. The job ID is handled in later processes if it cannot be resolved during scraping.	Instead of resolving the regex issue during later processes, the regex logic in the scraping process should be improved so that deduplication can also be handled during scraping. This would reduce the amount of recorded data during job listing scraping and lower memory usage.

Termination Logic Issues Although the maximum number of pages for a query is 167, some queries return fewer pages. Therefore, a termination logic had to be implemented. Initially, the logic terminated the scraping process after detecting three consecutive empty pages. However, the website did not behave in this way. Instead, when a requested page was unavailable, the website redirected the request to the first page. This resulted in duplicated data and wasted requests, especially for filters that returned less than one page of data, which is fewer than 30 records.	The first revision detected whether the number of job listings for three consecutive pages was the same and fewer than 27 records. This indicated that the job listing for the filter contained fewer than 30 records and there was no need to iterate through all 167 pages. However, tracking three consecutive pages was too conservative and could only effectively detect redirects when the filter returned a single page. Therefore, the updated termination logic terminates the scraping process for the combination as soon as any page returns fewer than 27 records, indicating the final page of the search results.	Instead of determining the final page based on the returned job count, it would be more efficient to capture the total number of available pages from the pagination section when the first page is requested and terminate the scraping process accordingly.
Insufficient data volume for the benchmarking phase Before implementing description scraping, the scraped data only consisted of job listings.	Job descriptions were scraped after job listing deduplication to increase the dataset size.	-
Increase processing time as the file size grows The problem occurred when compiling raw data files as the number of raw data files increased, and when scraping descriptions, as the descriptions added additional load to the dataframe.	A data/raw_data_compiled directory was introduced to move processed raw_data files so that they would not be reprocessed during subsequent compilation procedures. For description scraping, the dataset was also separated into records with and without descriptions.	-

Table 8: Challenges, solutions and future improvements

7.2 Limitations of the solution

- The solution depends heavily on human intervention, particularly when merging and separating files during the scraping process.
- Location attributes require hard-coded logic to identify states and countries due to inconsistent location inputs. As a result, some locations may not be correctly identified if they are not included in the predefined list.
- Salary attributes were also standardised using a hard-coded method, which may not always be accurate.
- The benchmarking attributes may be limited and may not fully represent the performance of the libraries across all evaluation aspects.

8. Conclusion and Future Work

In this project, a data processing pipeline was implemented, starting from web scraping from the Maukerja platform to data preprocessing, transformation and performance benchmarking across multiple data processing frameworks.

A large dataset consisting of 106,081 unique job postings was collected through an iterative scraping process.

Three data processing frameworks, namely Pandas, Polars, and PySpark, were evaluated using identical computational tasks, including filtering, aggregation and classification. The benchmarking results show clear performance differences among the framework. Polars achieved the best overall performance in terms of execution time and throughput, making it highly efficient for single-machine data processing tasks. The optimised Pandas implementation significantly improved performance compared to the baseline version, particularly in execution speed and memory efficiency. PySpark, while designed for distributed computing environments, exhibited higher execution time in a single-machine setup, indicating that its advantages are more apparent in cluster-based architectures.

For improvements that could be done in the future, the system can be extended to handle larger datasets to better evaluate PySpark's distributed processing capabilities in a multi-node cluster environment. Additionally, other data processing frameworks such as Dask, PyArrow, and Koalas could be included in the comparison.

9. References

GeeksforGeeks. (2026, January 13). *Implementing Web Scraping in Python with BeautifulSoup*.
GeeksforGeeks.
<https://www.geeksforgeeks.org/python/implementing-web-scraping-python-beautiful-soup/>

10. Appendices

Link to the repository: <https://github.com/drshahizan/HPDP/tree/main/2526/project/p1/TripleA>